데이터베이스(SQL)

1장. 설치와 실습 환경 구성

학습 목표

1) PostgreSQL(서버)과 DBeaver(클라이언트)의 역할을 구분한다.
2) PostgreSQL 18을 설치하고 DBeaver로 연결한다. (Windows / macOS)
3) 실습 데이터베이스 hd를 만들고 접속 대상을 hd로 바꾼다.
4) 데이터베이스와 스키마의 차이를 이해하고 mfg, pdm 스키마를 만든다.
5) Northwind는 스크립트로, Steel · AI4I · PdM은 CREATE TABLE 후 Import Data로 적재한다.
6) 적재 건수를 SQL로 조회해 실습 준비가 끝났는지 확인한다.

전체 흐름

1) 설치 → 2) DBeaver 연결 → 3) 데이터베이스 생성 → 4) 스키마 생성 → 5) 데이터 적재 → 6) 적재 건수 확인

01 실습 환경 구성

프로그램 구성

프로그램	역할
PostgreSQL	데이터베이스 본체 (데이터를 실제로 저장하고 처리하는 서버 프로그램)
DBeaver	PostgreSQL에 접속해 SQL을 실행하고 결과를 눈으로 확인하는 도구



PostgreSQL 설치 절차

Windows 설치

1) https://www.postgresql.org/download/windows/ 접속

2) "Download the installer" 클릭

3) 18 버전 다운로드클릭

4) 설치 파일 실행

macOS 설치

1) https://postgresapp.com/downloads.html 접속 (PostgreSQL 18 포함 버전 선택)

2) Postgres.app 다운로드 후 응용 프로그램(Applications) 폴더로 이동

3) 실행 후 "Initialize" 클릭

공식 문서: PostgreSQL 다운로드

Windows 설치 파일 다운로드 화면

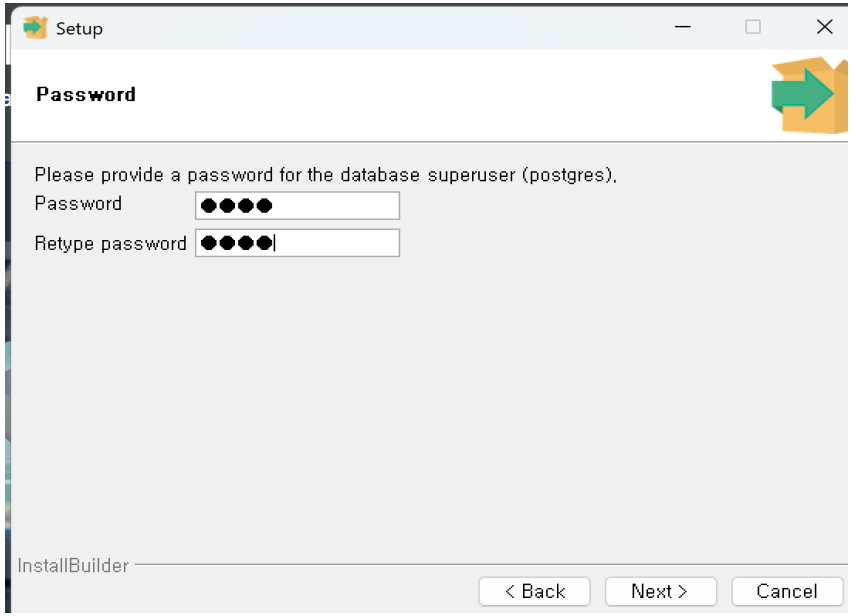
Download PostgreSQL

Open source PostgreSQL packages and installers from EDB

PostgreSQL Version	Linux x86-64	Linux x86-32	Mac OS X	Windows x86-64	Windows x86-32
18.4	postgresql.org	postgresql.org			Not supported
17.10	postgresql.org	postgresql.org			Not supported
16.14	postgresql.org	postgresql.org			Not supported
15.18	postgresql.org	postgresql.org			Not supported
14.23	postgresql.org	postgresql.org			Not supported

설치 중 선택 항목

항목	설정값
설치 경로	기본값 유지
설치 구성요소	기본값 유지
데이터 저장 경로	기본값 유지
비밀번호	1234
포트	5432 (기본값 유지)
로케일	Default locale



설치 구성요소에 pgAdmin 4와 Stack Builder도 표시될 수 있다. 이 수업은 DBeaver를 사용하므로 필수는 아니다

macOS 접속 정보

macOS 접속 정보 주의

Postgres.app 기본 로컬 설정에서는 비밀번호 없이 접속할 수 있다. 인증 방식을 변경했다면 해당 비밀번호를 입력한다.

항목	값
Host	localhost
Port	5432
User	macOS 사용자 이름
Password	기본 설정: 비워 둠

기본 설정에서는 current_user가 macOS 사용자 이름으로 나올 수 있다. 사용자와 인증 설정을 변경했다면 결과는 달라질 수 있다.

psql 설치 확인 (선택) — Windows

PostgreSQL 명령줄 도구(psql)로 설치를 확인할 수 있다. 이 과정의 실습은 DBeaver로 하므로, 이 단계를 건너뛰고 DBeaver 연결 후 SELECT version();으로 확인해도 된다.

psql을 어느 폴더에서나 실행하려면 PostgreSQL 실행 파일 폴더를 환경변수 Path에 등록해야 한다.

1) 시작 메뉴에서 "시스템 환경 변수 편집"을 검색해 연다.

2) 환경 변수 → 시스템 변수의 Path 선택 → 편집 → 새로 만들기

3) C:\Program Files\PostgreSQL\18\bin 을 추가하고 확인을 누른다.

4) 명령 프롬프트를 새로 열고 아래 명령을 실행한다.

```
psql --version
```

버전이 출력되면 psql 명령줄 도구의 실행 환경을 확인한 것이다. 서버 연결은 뒤에서 SELECT version();으로 확인한다.

psql 설치 확인 (선택) — macOS

터미널에서 아래 명령으로 Postgres.app의 명령줄 도구 경로를 등록한다. 비밀번호를 물으면 Mac 로그인 비밀번호를 입력한다.

```
sudo mkdir -p /etc/paths.d && echo /Applications/Postgres.app/Contents/Versions/latest/bin | sudo tee /etc/paths.d/postgresapp
```

터미널을 새로 열고 아래 명령을 실행한다.

```
psql --version
```

버전이 출력되면 psql 명령줄 도구의 실행 환경을 확인한 것이다. 서버 연결은 뒤에서 SELECT version();으로 확인한다.

DBeaver 설치

1) https://dbeaver.io/download/ 접속

2) DBeaver Community 버전 다운로드 (Windows는 Installer, macOS는 dmg)

3) 설치 후 실행

Windows
Installer

macOS
dmg

공식 문서: DBeaver 연결 만들기(Create connection) / DBeaver PostgreSQL 드라이버

PostgreSQL 연결 만들기

1) 데이터베이스→ New Database Connection

2) PostgreSQL 선택 → Next

3) Main 탭 입력

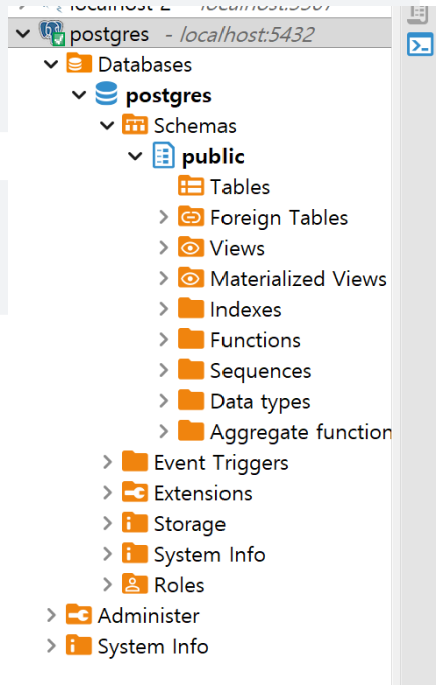
항목	Windows	macOS (Postgres.app)
Host	localhost	localhost
Port	5432	5432
Database	postgres	postgres
Username	postgres	macOS 사용자 이름
Password	설치할 때 정한 비밀번호	비워 둔다

Show all databases와 연결 테스트

4) Main 탭의 Database 칸 옆 **Show all databases**를 체크한다. 체크하면 왼쪽 Database Navigator에 서버의 모든 데이터베이스(postgres, hd 등)가 보인다.

5) Test Connection 클릭

- 처음 연결할 때 PostgreSQL 드라이버를 내려받을지 묻는다. Download를 누른다. 인터넷 연결이 필요하다.
- macOS는 처음 연결할 때 Postgres.app이 DBeaver의 접속을 허용할지 묻는 창이 뜰 수 있다. 허용한다.



6) Connected가 나오면 Finish

SQL 실행 방법

연결 이름 우클릭 → SQL Editor → New SQL script로 SQL 편집기를 연다.

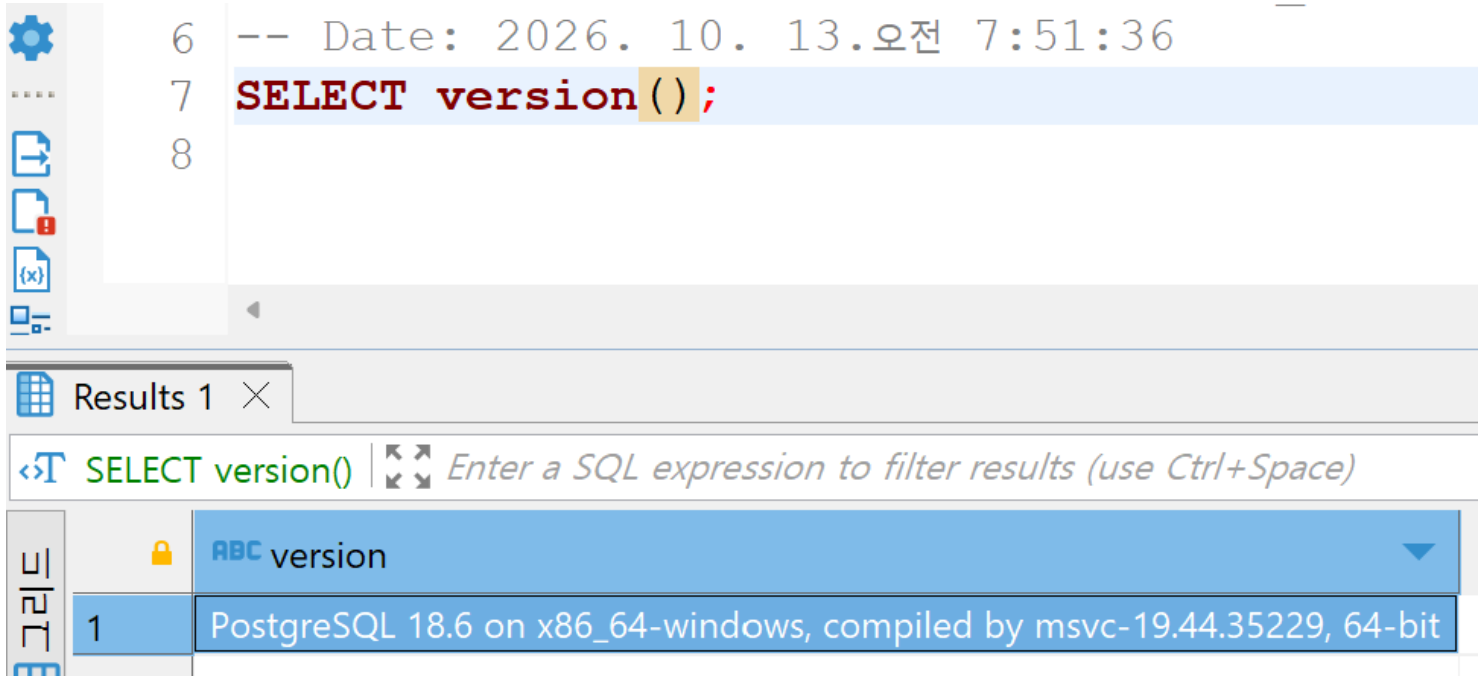
동작	Windows	macOS	설명
새 SQL 스크립트	Ctrl +]	Control +]	선택한 연결의 새 편집기를 연다
SQL 문 실행	Ctrl + Enter	Cmd + Enter	커서가 있는 문장 하나(또는 드래그한 구간)만 실행한다
스크립트 실행	Alt + X	Option + X	선택 영역이 있으면 선택한 SQL을, 없으면 편집기 전체를 순서대로 실행한다

공식 문서: DBeaver Shortcuts

설치와 연결 확인

```
SELECT version();
```

PostgreSQL 버전이 출력되면 설치와 연결이 모두 완료된 것이다.



02 데이터베이스와 스키마

실습 데이터베이스 hd 생성

postgres 데이터베이스에 접속한 상태에서 SQL 편집기를 열고 실행한다.

```
CREATE DATABASE hd;
```

CREATE DATABASE는 트랜잭션 블록 안에서 실행할 수 없다.
DBeaver에서 Auto-Commit이 켜져 있는지 확인하고 단독으로 실행한다.

PostgreSQL에는 CREATE ROLE, CREATE TABLESPACE, CREATE DATABASE등은 IF NOT EXISTS 문법이 없다.
hd가 이미 있으면 오류가 나므로 다시 실행하지 않는다.

공식 문서: CREATE DATABASE(한국어 번역, 13 버전 기준) / CREATE DATABASE(18 버전 원문)

접속 대상 변경

hd를 만든 뒤 접속 대상을 hd로 바꾼다.

1) 왼쪽 Database Navigator에서 연결 이름 우클릭 → Edit Connection

2) Main 탭의 Database 칸을 postgres → hd로 변경

3) OK → 다시 연결할지 물으면 Yes

4) 연결 이름 우클릭 → SQL Editor → New SQL script로 새 편집기를 연다.

```
SELECT current_database();
```

결과 **hd**

데이터베이스 · 스키마 · 테이블의 계층 구조

PostgreSQL 서버
ㄴ 데이터베이스 (hd)
 ├─ 스키마 (public)
 | ├─ 테이블 (customers)
 | ㄴ 테이블 (orders)
 ├─ 스키마 (mfg)
 | ├─ 테이블 (tb_steel)
 | ㄴ 테이블 (tb_ai4i)
 ㄴ 스키마 (pdm)
 ㄴ 테이블 (pdm_failures)

스키마(Schema) - **테이블, 뷰, 함수를 담는 논리적 구분 단위.**

스키마 안에 다른 스키마를 만들 수 없다

공식 문서: Schemas(18버전 원문)

데이터베이스와 스키마의 차이

데이터베이스와 스키마는 둘 다 객체를 담는 단위이지만 분리 방식이 다르다.

	데이터베이스가 다르면	스키마가 다르면
한 번에 접속	하나만 가능	제한 없음
함께 조회	직접 조회 불가	가능
JOIN	직접 JOIN 불가	가능

하나의 연결은 하나의 데이터베이스에만 접속한다. 반면 그 데이터베이스 안의 스키마는 권한만 있으면 모두 접근할 수 있다.

데이터베이스구조

데이터베이스 hd

- 스키마 public
- 스키마 mfg
- 스키마 pdm

public 스키마

기본 설치 환경에서는 스키마를 지정하지 않고 만든 테이블이 보통 **public** 스키마에 들어간다.

새 데이터베이스에는 기본적으로 public 스키마가 존재한다.

스키마 구성

스키마	데이터셋	테이블 수
public	Northwind 주문 데이터	14
mfg	AI4I 제조 품질, Steel 에너지	2
pdm	Azure 예지보전	5

mfg는 manufacturing(제조), pdm은 predictive maintenance(예지보전)의 약자다.

스키마 생성

```
CREATE SCHEMA IF NOT EXISTS mfg;
CREATE SCHEMA IF NOT EXISTS pdm;
```

public은 이미 존재하므로 만들지 않는다.

IF NOT EXISTS는 스키마가 이미 존재하면 새로 만들지 않고 알림만 출력한다.

공식 문서: CREATE SCHEMA(한국어 번역, 13 버전 기준) / CREATE SCHEMA(18 버전 원문)

스키마 생성 결과 확인

```
-- 생성 결과 확인
SELECT schema_name AS 스키마명
FROM information_schema.schemata
WHERE schema_name IN ('public', 'mfg', 'pdm')
ORDER BY schema_name;
```

결과

스키마명
mfg
pdm
public

왼쪽 Database Navigator에 mfg, pdm이 보이지 않으면 hd → Schemas 우클릭 → Refresh(F5)를 누른다.

스키마를 지정한 테이블 생성과 조회

테이블 이름 앞에 **스키마명.**을 붙인다.

```
-- 생성
CREATE TABLE mfg.tb_ai4i (
udi INTEGER PRIMARY KEY,
product_type CHAR(1) NOT NULL
);

-- 조회
SELECT * FROM mfg.tb_ai4i LIMIT 5;
```

데이터베이스와 스키마 핵심 정리

1) 데이터베이스 안에 스키마가 있고, 스키마 안에 테이블이 있다.
2) 하나의 연결은 하나의 데이터베이스에만 접속한다. 데이터베이스가 다르면 함께 조회할 수 없다.
3) 스키마가 다른 테이블은 함께 조회하고 JOIN할 수 있다.
4) 기본 설치 환경에서는 스키마를 지정하지 않고 만든 테이블이 보통 public에 들어간다. 정확히는 search_path에서 처음 사용 가능한 스키마에 생성된다.
5) 이 과정은 hd 데이터베이스 하나에 public, mfg, pdm 세 스키마를 사용한다.

03 Northwind 적재

실습 데이터셋 구성

데이터셋	주요 학습 범위	출처
Northwind	조회, 조건, 정렬, 집계, 조인	Microsoft 예제 DB의 PostgreSQL 변환본
Steel Industry	제조 에너지 사용량과 시간 데이터 분석	Steel Industry Energy Consumption
AI4I 2020	제조 데이터 집계와 고장 분석	UCI Machine Learning Repository
Microsoft PdM	시계열, 센서, 정비 및 고장 이력 분석	Microsoft 예지보전 예제

공식 문서: 새 테이블 만들기(한국어 번역, 13 버전 기준) / Creating a New Table(18 버전 원문)

데이터 적재 방법

데이터 적재 방법은 두 가지다.

Northwind

northwind.sql 파일 하나에 테이블 생성문과 데이터 입력문이 모두 들어 있다. DBeaver에서 스크립트로 실행한다.

Steel, AI4I, PdM

CREATE TABLE로 테이블을 먼저 만들고, DBeaver의 **Import Data**로 CSV를 넣는다.

Northwind 개요와 다운로드

Northwind는 가상의 무역회사인 Northwind Traders의 고객, 주문, 제품, 직원, 공급업체 데이터를 모델링한 예제 데이터베이스이다.

PostgreSQL용 변환본을 사용한다.

```
https://github.com/pthom/northwind_psql
```

다운로드 방법

1) 저장소에 접속한다.

2) northwind.sql 파일을 선택한다.

3) Raw 파일을 저장하거나 저장소 전체를 ZIP으로 내려받는다.

4) ZIP 파일을 내려받았다면 압축을 해제한다.

공식 문서: northwind_psql 저장소

DBeaver에서 Northwind 적재

동작	Windows	macOS
1) 연결 이름 우클릭 → SQL Editor → New SQL script로 새 편집기를 연다.	Ctrl +]	Control +]
2) northwind.sql 파일을 메모장(macOS는 텍스트 편집기)으로 열어 전체 선택 후 복사한다.	Ctrl + A → Ctrl + C	Cmd + A → Cmd + C
3) DBeaver 새 편집기에 붙여 넣는다.	Ctrl + V	Cmd + V
4) 스크립트 실행으로 전체를 실행한다.	Alt + X	Option + X

Northwind 적재 시 주의

문장 실행이 아니라 스크립트 실행

Ctrl + Enter(macOS Cmd + Enter)는 커서가 있는 문장 하나만 실행한다. 여기서는 반드시 스크립트 실행으로 전체를 실행한다.

테이블 생성과 CSV Import가 따로 필요 없음

northwind.sql 파일에는 테이블 생성문과 데이터 입력문이 함께 포함되어 있으므로 Northwind 테이블을 따로 생성하거나 CSV를 개별 Import할 필요가 없다.

Northwind 주요 테이블

테이블	한글 설명
customers	고객 기본 정보
orders	주문 기본 정보
order_details	주문별 제품, 수량 및 가격
products	제품 기본 정보
categories	제품 분류
employees	직원 정보
suppliers	공급업체 정보
shippers	배송업체 정보

Northwind 적재 건수 확인

대표 배포본의 예상 결과

테이블	예상 건수
customers	91
orders	830
order_details	2155
products	77

한 번에 확인하려면 다음 SQL을 사용한다.

```
SELECT
(SELECT COUNT(*) FROM public.customers) AS customers,
(SELECT COUNT(*) FROM public.orders) AS orders,
(SELECT COUNT(*) FROM public.order_details) AS order_details,
(SELECT COUNT(*) FROM public.products) AS products;
```

04 Steel Industry 적재

Steel Industry 데이터

Steel Industry 데이터는 철강 제조 공정의 시간대별 **에너지 사용량(kWh)**, 무효전력량(kVarh),부하 유형 등을 기록한 데이터이다.

시간대별 전력 소비 분석, 요일별 비교, 부하 유형별 집계 등에 사용할 수 있다.

원출처

https://archive.ics.uci.edu/dataset/851/steel+industry+energy+consumption

공식 문서: CREATE TABLE(한국어 번역, 13 버전 기준) / CREATE TABLE(18 버전 원문)

Steel 테이블 생성

hd에 접속한 DBeaver SQL 편집기에서 실행한다.

```
DROP TABLE IF EXISTS mfg.tb_steel;

CREATE TABLE mfg.tb_steel (
measured_at VARCHAR(16) NOT NULL,
usage_kwh NUMERIC(10, 2) NOT NULL,
lagging_current_reactive_power_kvarh NUMERIC(10, 2) NOT NULL,
leading_current_reactive_power_kvarh NUMERIC(10, 2) NOT NULL,
co2_tco2 NUMERIC(6, 2) NOT NULL,
lagging_current_power_factor NUMERIC(6, 2) NOT NULL,
leading_current_power_factor NUMERIC(6, 2) NOT NULL,
nsm INTEGER NOT NULL,
week_status VARCHAR(7) NOT NULL,
day_of_week VARCHAR(9) NOT NULL,
load_type VARCHAR(12) NOT NULL
);
```

여러 문장을 한 번에 실행하므로 스크립트 실행 (Windows Alt + X / macOS Option + X)을 사용한다.

Steel CSV 컬럼과 테이블 컬럼 대응 (1/2)

DBeaver Import는 **CSV 첫 줄의 열 이름**으로 테이블 컬럼을 찾는다. 이름이 다른 4개(굵게 표시)는 Import할 때 직접 연결해야 한다.

순서	CSV 원본 열	테이블 컬럼	한글 설명
1	date	measured_at	에너지 사용량 측정 일시
2	Usage_kWh	usage_kwh	에너지 사용량(kWh)
3	Lagging_Current_Reactive.Power_kVarh	lagging_current_reactive_power_kvarh	지상 무효전력량(kVarh)
4	Leading_Current_Reactive_Power_kVarh	leading_current_reactive_power_kvarh	진상 무효전력량(kVarh)
5	CO2(tCO2)	co2_tco2	CO2 관련 값(원본 열 이름: CO2(tCO2))
6	Lagging_Current_Power_Factor	lagging_current_power_factor	지상 전류 역률(%)

Steel CSV 컬럼과 테이블 컬럼 대응 (2/2)

순서	CSV 원본 열	테이블 컬럼	한글 설명
7	Leading_Current_Power_Factor	leading_current_power_factor	진상 전류 역률(%)
8	NSM	nsm	자정 이후 경과 시간(초)
9	WeekStatus	week_status	평일 또는 주말 구분
10	Day_of_week	day_of_week	요일
11	Load_Type	load_type	부하 유형

굵게 표시한 열(date, Lagging_Current_Reactive.Power_kVarh, CO2(tCO2), WeekStatus)은 Import할 때 직접 연결한다.

Steel 단위 읽기와 의미

kWh — 킬로와트시

일정 시간 동안 사용한 전기에너지의 양을 나타낸다. 예를 들어 1 kW의 전력을 1시간 사용하면 1 kWh이다.

kVarh — 킬로바르시

(kilovolt-ampere reactive hour) 일정 시간 동안 누적된 **무효전력량**을 나타낸다. 모터·변압기 같은 교류 부하에서 전기장·자기장을 형성하기 위해 오가는 성분의 누적량으로 이해하면 된다.

% — 퍼센트

100을 기준으로 나타낸 비율이다. 이 데이터에서는 역률을 백분율로 표현할 때 사용한다.

s — 초(second, 세컨드)

시간의 단위이다. NSM은 **Number of Seconds from Midnight**, 즉 자정 이후 몇 초가 지났는지를 의미한다.

단위 기호를 읽는 것과 값의 의미를 함께 기억한다. 예를 들어 usage_kwh = 50은 단순히 "50"이 아니라 **에너지 사용량 50 kWh**라는 뜻이다.

Steel 한글 컬럼 설명 저장

```
COMMENT ON TABLE mfg.tb_steel
IS '철강 산업 시간대별 에너지 사용 데이터';

COMMENT ON COLUMN mfg.tb_steel.measured_at
IS '에너지 사용량 측정 일시';

COMMENT ON COLUMN mfg.tb_steel.usage_kwh
IS '에너지 사용량(kWh)';

COMMENT ON COLUMN mfg.tb_steel.lagging_current_reactive_power_kvarh
IS '지상 무효전력량(kVarh)';

COMMENT ON COLUMN mfg.tb_steel.leading_current_reactive_power_kvarh
IS '진상 무효전력량(kVarh)';
```

공식 문서: COMMENT(18 버전 원문)

1) Import 시작

DBeaver에서 CSV를 적재한다. Database Navigator에서 아래 순서로 찾아간다.

hd → **Schemas** → **mfg** → **Tables** → **tb_steel** → **마우스 오른쪽 버튼** → **Import Data**

> tb_steel이 보이지 않으면 Tables 우클릭 → Refresh(F5)를 먼저 누른다.

공식 문서: DBeaver Data import / DBeaver Tables mapping(Data migration)

2) 형식과 파일 선택

CSV 선택 → Next → 수업 배포본 Steel_industry_data.csv 선택 후 Importer settings를 확인한다.

항목	값
Encoding	UTF-8
Column delimiter	,
Header position	top
Quote char	"

수업 배포 CSV가 UTF-8인지 확인한다. 다른 인코딩으로 읽으면 열 이름이 깨져 매핑에 문제가 생길 수 있다.

Header position은 top으로 둔다. none으로 두면 CSV 첫 줄(열 이름)이 데이터로 들어가 오류가 난다.

3) Tables mapping 확인

Next를 누르면 Tables mapping 화면이 나온다.

화면 위쪽 **Target container가 hd.mfg인지** 확인한다.

Steel_industry_data.csv 행의 **Target이 tb_steel이고 Mapping이 existing인지** 확인한다.

Target이 [skip]이면 Target 칸을 클릭해 목록에서 tb_steel을 고른다. **글자를 직접 입력하면 그 이름으로 새 테이블이 만들어지므로 반드시 목록에서 고른다.**

Target container: **hd.mfg**

Source	Target	Mapping
Steel_industry_data.csv	tb_steel	existing

4) Columns mapping 확인 값

직접 연결하는 4개 열 (나머지 7개 열도 Mapping은 existing)

Source Column	Target Column	Mapping
date	measured_at	existing
Lagging_Current_Reactive.Power_kVarh	lagging_current_reactive_power_kvarh	existing
CO2(tCO2)	co2_tco2	existing
WeekStatus	week_status	existing

Mapping 값	의미	결과
existing	기존 컬럼에 연결	정상
skip	이 열은 넣지 않음	해당 컬럼이 비어 NOT NULL 오류
new	같은 이름의 새 컬럼을 테이블에 추가	테이블 구조가 바뀌고 원래 컬럼은 비어 NOT NULL 오류

5) 실행과 오류 대처

OK → Next → Data load settings는 기본값 그대로 Next → Proceed

아래쪽 **Drop and create the table은 체크하지 않는다.** 체크하면 tb_steel이 지워지고 CSV 열 이름 그대로 새 테이블이 만들어진다.

오류가 나면

Import가 한 번 실패하면 잘못된 매핑 때문에 테이블에 컬럼이 추가되었을 수 있다. 「Steel 테이블 생성」의 DROP TABLE부터 「Steel 한글 컬럼 설명 저장」의 COMMENT까지 다시 실행하고, Tables를 Refresh한 뒤 Import를 처음부터 다시 한다.

오류 메시지	원인
null 값이 not null 제약조건을 위반했습니다	어떤 열이 기존 컬럼에 연결되지 않았다 (skip 또는 new)
INSERT INTO ... DEFAULT VALUES	모든 열이 skip 상태다
컬럼 이름 앞에 癤 같은 깨진 글자	Encoding이 UTF-8이 아니다

Steel 적재 결과 확인

```
SELECT COUNT(*) AS 전체건수
FROM mfg.tb_steel;
```

예상 결과

35040

```
SELECT *
FROM mfg.tb_steel
LIMIT 5;
```

05 AI4I 2020 적재

AI4I 2020 데이터

AI4I 2020 Predictive Maintenance Dataset은 제조 설비의 온도, 회전속도, 토크, 공구 마모도와 고장 여부를 포함한 합성 데이터셋이다.

합성 데이터는 실제 산업 데이터의 특성을 반영하여 인공적으로 생성한 데이터이다. 실제 제조 데이터는 보안문제로 공개하기 어렵기 때문에 연구와 교육 목적으로 활용된다.

원출처 · 실습 파일 ai4i2020.csv

https://archive.ics.uci.edu/dataset/601/ai4i+2020+predictive+maintenance+dataset

AI4I 테이블 생성

```
DROP TABLE IF EXISTS mfg.tb_ai4i;

CREATE TABLE mfg.tb_ai4i (
udi INTEGER PRIMARY KEY,
product_id VARCHAR(20) NOT NULL,
product_type CHAR(1) NOT NULL,
air_temperature_k NUMERIC(5, 1) NOT NULL,
process_temperature_k NUMERIC(5, 1) NOT NULL,
rotational_speed_rpm INTEGER NOT NULL,
torque_nm NUMERIC(5, 1) NOT NULL,
tool_wear_min INTEGER NOT NULL,
machine_failure SMALLINT NOT NULL,
twf SMALLINT NOT NULL,
hdf SMALLINT NOT NULL,
pwf SMALLINT NOT NULL,
```

AI4I CSV 컬럼과 테이블 컬럼 대응 (1/2)

CSV 원본 열	테이블 컬럼	한글 설명
UDI	udi	행을 구분하는 고유 번호
Product ID	product_id	제품 식별 번호
Type	product_type	제품 품질 유형(L, M, H)
Air temperature [K]	air_temperature_k	공기 온도(K)
Process temperature [K]	process_temperature_k	공정 온도(K)
Rotational speed [rpm]	rotational_speed_rpm	회전 속도(rpm)
Torque [Nm]	torque_nm	토크(Nm)

AI4I CSV 컬럼과 테이블 컬럼 대응 (2/2)

CSV 원본 열	테이블 컬럼	한글 설명
Tool wear [min]	tool_wear_min	공구 마모 시간(분)
Machine failure	machine_failure	전체 설비 고장 여부
TWF	twf	공구 마모 고장 여부 (Tool Wear Failure)
HDF	hdf	열 방출 고장 여부 (Heat Dissipation Failure)
PWF	pwf	전력 고장 여부 (Power Failure)
OSF	osf	과부하 고장 여부 (Overstrain Failure)
RNF	rnf	임의 고장 여부 (Random Failure)

AI4I 단위 읽기와 의미

K — 켈빈

절대온도의 단위이다. 298.1 K는 "298.1 켈빈"이라고 읽는다. 섭씨 온도로 바꾸려면 273.15를 뺀다.

rpm — 알피엠

(revolutions per minute) **1분 동안 회전한 횟수**를 뜻한다. 회전 속도를 나타낸다.

Nm — 뉴턴미터

(newton metre) 이 데이터에서는 **토크(회전시키는 힘의 효과)**를 나타내는 단위이다.

min — 분

(minute) 시간 단위이다. tool_wear_min은 공구가 사용되어 마모된 누적 시간을 분 단위로 나타낸다.

1. 온도의 3대 단위 비교

- **섭씨 (°C):** 우리나라에서 일상적으로 쓰는 단위 (물이 어는 점: 0°C)
- **화씨 (°F):** 미국 등에서 주로 쓰는 단위 (물이 어는 점: 32°F)
- **켈빈 (K):** 과학과 공학에서 사용하는 **절대온도** 단위 (물이 어는 점: 273.15 K)

2. 섭씨와 켈빈의 관계

- **온도의 간격(1도의 크기)은 동일**
- **시작하는 기준점**이 다름
- 섭씨온도에 **273.15**를 더하면 켈빈온도가 됨

$$\text{켈빈}(\mathrm{K}) = \text{섭씨}(^{\circ}\mathrm{C}) + 273.15$$

AI4I CSV 적재

Steel 적재와 같은 순서로 진행한다. 다른 점만 정리한다.

hd → **Schemas** → **mfg** → **Tables** → **tb_ai4i** → **마우스 오른쪽 버튼** → **Import Data**

단계	확인할 것
파일 선택	ai4i2020.csv, Encoding UTF-8, Header position top
Tables mapping	Target container hd.mfg, Target tb_ai4i, Mapping existing
Columns mapping	「AI4I CSV 컬럼과 테이블 컬럼 대응」 표대로 14개 열을 연결하고 전부 existing인지 확인
실행	Drop and create the table 체크하지 않음 → Proceed

오류가 나면 「AI4I 테이블 생성」의 DROP TABLE부터 다시 실행하고 Import를 처음부터 다시 한다.

공식 문서: DBeaver Data import

AI4I 적재 확인

전체 건수

```
SELECT COUNT(*) AS total_count FROM mfg.tb_ai4i;
```

예상 결과

10000

제품 품질 유형별 건수

```
SELECT
product_type,
COUNT(*) AS product_count
FROM mfg.tb_ai4i
GROUP BY product_type
ORDER BY product_type;
```

대표 데이터의 예상 결과

product_type	product_count
H	1003
L	6000
M	2997

전체 고장 건수

```
SELECT SUM(machine_failure) AS failure_count FROM mfg.tb_ai4i;
```

예상 결과

339

06 PdM 적재와 전체 확인

Microsoft PdM 데이터

Microsoft PdM 데이터는 설비 100대의 센서 측정값, 에러, 정비, 부품 고장 이력을 포함한다. 텔레메트리는 2015년의 시간별 측정값이며, 정비 이력에는 2014년과 2015년 기록이 함께 포함된다.

텔레메트리는 설비 또는 장치에서 원격으로 수집한 측정값이다. 이 데이터에서는 전압, 회전, 압력, 진동 값이 **시간별 평균값**으로 기록되어 있다.

> 예지보전(Predictive Maintenance, PdM)은 설비의 센서 데이터와 고장 이력을 분석하여 고장이 발생하기 전에 이상 가능성을 예측하고 정비하는 방식이다.

원출처(참고)
https://github.com/microsoft/sqlworkshops/tree/master/SQLServerAndAzureMachineLearning/ML%20Services%20for%20SQL%20Server/data

PdM 원본 파일과 테이블

CSV 파일	테이블	한글 설명
PdM_machines.csv	pdm_machines	설비 모델과 사용 연수
PdM_telemetry.csv	pdm_telemetry	시간별 센서 측정값
PdM_errors.csv	pdm_errors	설비 에러 발생 이력
PdM_maint.csv	pdm_maint	부품 정비 및 교체 이력
PdM_failures.csv	pdm_failures	부품 고장 이력

PdM 테이블 생성

```
DROP TABLE IF EXISTS pdm.pdm_telemetry;
DROP TABLE IF EXISTS pdm.pdm_maint;
DROP TABLE IF EXISTS pdm.pdm_errors;
DROP TABLE IF EXISTS pdm.pdm_failures;
DROP TABLE IF EXISTS pdm.pdm_machines;

CREATE TABLE pdm.pdm_machines (
machineid INTEGER PRIMARY KEY,
model VARCHAR(20),
age INTEGER
);
```

PdM 컬럼 설명 (1/2)

pdm_machines

컬럼	한글 설명
machineid	설비 고유 번호
model	설비 모델
age	설비 사용 연수

pdm_telemetry

컬럼	한글 설명
datetime	센서 측정 일시
machineid	설비 고유 번호
volt	전압 측정값
rotate	회전 측정값
pressure	압력 측정값
vibration	진동 측정값

PdM 컬럼 설명 (2/2)

pdm_errors

컬럼	한글 설명
datetime	에러 발생 일시
machineid	설비 고유 번호
errorid	에러 유형

pdm_maint

컬럼	한글 설명
datetime	정비 또는 부품 교체 일시
machineid	설비 고유 번호
comp	정비한 부품 유형

pdm_failures

컬럼	한글 설명
datetime	고장 발생 일시
machineid	설비 고유 번호
failure	고장난 부품 유형

PdM CSV 적재

1. PdM_machines.csv → pdm_machines
2. PdM_failures.csv → pdm_failures
3. PdM_errors.csv → pdm_errors
4. PdM_maint.csv → pdm_maint
5. PdM_telemetry.csv → pdm_telemetry

PdM Import 확인 항목

단계	확인할 것
Import 시작	hd → Schemas → pdm → Tables → 테이블 우클릭 → Import Data
파일 선택	Encoding UTF-8, Header position top
Tables mapping	Target container가 hd.pdm인지 확인한다. hd.public이면 잘못 들어간 것이다
Columns mapping	CSV 열 이름과 테이블 컬럼명이 같다(machineID → machineid처럼 대소문자만 다르다). 전부 existing인지만 확인한다
실행	Drop and create the table 체크하지 않음 → Proceed

pdm_telemetry는 87만여 건이라 적재에 시간이 오래 걸린다. 오른쪽 아래 진행 표시가 끝나고 완료 알림이 뜰 때까지 기다린다.

PdM 적재 확인

```
SELECT COUNT(*) AS machine_count FROM pdm.pdm_machines;
SELECT COUNT(*) AS failure_count FROM pdm.pdm_failures;
SELECT COUNT(*) AS error_count FROM pdm.pdm_errors;
SELECT COUNT(*) AS maint_count FROM pdm.pdm_maint;
SELECT COUNT(*) AS telemetry_count FROM pdm.pdm_telemetry;
```

전체 적재 기준 건수

데이터셋	대표 건수
Northwind - customers	91
Steel	35,040
AI4I	10,000
PdM - machines	100
PdM - failures	761
PdM - errors	3,919
PdM - maint	3,286
PdM - telemetry	배포 CSV 실제 건수

정리

챕터	핵심
01 실습 환경 구성	PostgreSQL 서버 + DBeaver 클라이언트, localhost · 5432로 연결, SELECT version();으로 확인
02 데이터베이스와 스키마	hd 하나에 public · mfg · pdm 세 스키마
03 Northwind 적재	northwind.sql을 스크립트 실행(Alt + X / Option + X)으로 public에 적재
04 Steel Industry 적재	CREATE TABLE → COMMENT → Import Data, 35040건
05 AI4I 2020 적재	CHECK 제약조건이 있는 테이블, 10000건 · 고장 339건
06 PdM 적재와 전체 확인	CSV 5개를 machines부터 적재, telemetry는 배포 CSV 실제 건수 기준